Slovenská Technická Univerzita v Bratislave

Fakulta informatiky a informačných technológií

Komunikačný protokol pre IoT meteorologické zariadenia

Počítačové a komunikačné siete

Sophie Hetešová

AIS ID: 141435

Vedúci cvičenia: Bc. Filip Hromada
Čas cvičenia: Štvrtok – 8:00

Obsah

Úvod a architektúra **2**

Použité knižnice 2

Návrh protokolu **3**

Relácia a polia správ 3

Typy správ a príklady 3

Parametre senzorov 4

Integrita a časovanie 5

Komunikačné cykly **6**

Prijímanie na serveri 6

Odosielanie na serveri 7

Odosielanie vo vlákne senzora 8

Prijímanie vo vlákne senzora 9

Sekvenčné diagramy a overenie UAT **10**

Návrh merania efektivity **14**

Zhodnotenie **14**

Úvod a architektúra

Riešenie prenáša meteorologické merania vlastným aplikačným protokolom nad UDP/IPv4. Správy sú objekty JSON zakódované v UTF-8; jeden datagram obsahuje jednu správu. Protokol zabezpečuje registráciu senzorov, potvrdenie meraní, kontrolu integrity a dostupnosti.

- `server.py` registruje senzory, prideľuje tokeny, overuje merania a spravuje časovače relácií.
- `tester.py` simuluje ThermoNode (T-01), WindSense (W-01), RainDetect (R-01) a AirQuality (A-01). Umožňuje vlastné merania, poškodenie správy, prepínanie automatiky, odpojenie a novú registráciu.
- `lib.py` obsahuje spoločné funkcie pre JSON, CRC32, validáciu, logovanie a terminálové rozhranie.

Každý senzor má vlastné vlákno, UDP socket, frontu príkazov a časovače. Hlavné vlákno testera obsluhuje menu; požiadavky odovzdáva príslušnému senzoru cez frontu. Čakanie jedného senzora neblokuje ostatné. Server spracúva komunikáciu a relácie v jednej sieťovej slučke, jeho menu beží v pomocnom vlákne.

Programy sa spúšťajú príkazmi `python3 server.py` a `python3 tester.py`. IP adresa a port sa zadávajú pri spustení, predvolene 127.0.0.1:9000. UDP sockety sa vytvárajú a viažu priamo knižnicou `socket`.

Použité knižnice

Aplikácia používa iba štandardnú knižnicu Pythonu:

Moduly	**Úloha**
`socket`	UDP komunikácia.
`threading`, `queue`	Vlákna a odovzdávanie príkazov.
`json`, `zlib`	Formát správ a CRC32.
`ipaddress`, `decimal`	Validácia adries a číselných vstupov.
`secrets`, `random`, `copy`	Tokeny, náhodné merania a poškodená kópia.
`time`, `logging`	Časovače a logovanie.
`os`, `sys`, `shutil`, `readline`	Terminál; `readline` je voliteľný.

Logy udalostí (SYSTEM) a prenosov (MESSAGE) sa zapisujú do `server.log` a `tester.log`; pri novom spustení sa prepíšu. Súbor zachováva celý obsah správ a formát `čas | typ | úroveň | zdroj | udalosť`. Čas je UTC s milisekundami. Konzola zobrazuje prehľadnejšie bloky s farbami a jednotkami.

Návrh protokolu

Relácia a polia správ

REGISTER obsahuje typ a jedinečné ID senzora. Server uloží jeho zdrojovú IP adresu a port a pridelí token: osem náhodných bajtov zapísaných ako 16 hexadecimálnych znakov. Ďalšie merania sa overujú podľa ID, tokenu a zdrojovej adresy. Po zrušení relácie token prestáva platiť. Typ DATA sa určuje z registrácie.

Pole	Význam
`type`	Reťazec označujúci typ správy.
`seq`	Kladné celé poradové číslo správy odosielateľa.
`ack_for`	Celé číslo správy, na ktorú odpoveď reaguje.
`sensor_id`	Reťazec s jedinečným ID senzora.
`sensor_type`	Reťazec s typom senzora.
`token`	Reťazec identifikujúci reláciu.
`values`	Objekt s hodnotami všetkých parametrov senzora.
`crc`	Nezáporný 32-bitový kontrolný súčet DATA.
`reason`	Textový dôvod odmietnutia alebo retransmisie.

Každý senzor má vlastný čítač `seq` od hodnoty 1; server používa svoj čítač. Odpoveď sa priraďuje cez socket senzora a `ack_for`. Nová správa má nové číslo, retransmisia zachováva pôvodné.

Typy správ a príklady

Príklady používajú ukážkový token. Zalomenia slúžia na čitateľnosť; program posiela JSON bez nadbytočných medzier.

REGISTER – senzor → server. Žiadosť o registráciu typu a ID. Už obsadené ID sa odmietne.

```
{"type":"REGISTER","seq":1,"sensor_type":"ThermoNode",
 "sensor_id":"T-01"}
```

ACK – server → senzor. Potvrdzuje registráciu alebo platné meranie. Token sa pridáva iba pri registrácii.

```
{"type":"ACK","seq":1,"ack_for":1,
 "token":"a1b2c3d4e5f60718"}
{"type":"ACK","seq":2,"ack_for":2}
```

DATA – senzor → server. Automatické aj ručné meranie majú rovnaký formát. Objekt `values` obsahuje parametre registrovaného typu.

```
{"type":"DATA","seq":2,"sensor_id":"T-01",
 "token":"a1b2c3d4e5f60718","values":{
 "temperature":21.3,"humidity":55.4,
 "dew_point":11.9,"pressure":1013.25},"crc":1819254021}
```

RETRANSMIT – server → senzor. Pri chybnom CRC server vyžiada správny originál. Tester overí reláciu a znova odošle tú istú DATA s rovnakým `seq`, hodnotami a CRC. Limit sú tri odoslania vrátane pôvodného.

```
{"type":"RETRANSMIT","seq":3,"ack_for":2,
 "sensor_id":"T-01","token":"a1b2c3d4e5f60718",
 "reason":"Integrity check failed"}
```

REJECT – server → senzor. Odmietnutie napríklad pri neplatnom tokene, obsadenom ID alebo hodnotách mimo rozsahu.

```
{"type":"REJECT","seq":4,"ack_for":2,
 "reason":"Invalid sensor ID or token"}
```

KEEPALIVE – server → senzor. Overuje dostupnosť senzora, od ktorého neprichádzajú merania.

```
{"type":"KEEPALIVE","seq":5,"sensor_id":"T-01",
 "token":"a1b2c3d4e5f60718"}
```

KEEPALIVE_ACK – senzor → server. Odpoveď na konkrétnu sondu; posiela sa aj pri vypnutej automatike.

```
{"type":"KEEPALIVE_ACK","seq":3,"ack_for":5,
 "sensor_id":"T-01","token":"a1b2c3d4e5f60718"}
```

Parametre senzorov

Obe hranice rozsahov sú povolené. Uvedená desatinná presnosť je maximálna; celočíselné polia musia byť celé čísla JSON. Tester aj server hodnoty validujú. Neplatný vstup možno opraviť bez pádu programu.

Parameter	Rozsah	Presnosť
ThermoNode		
`temperature`	−50,0 až 60,0 °C	1 desatinné miesto
`humidity`	0,0 až 100,0 %	1 desatinné miesto
`dew_point`	−50,0 až 60,0 °C	1 desatinné miesto
`pressure`	800,00 až 1100,00 hPa	2 desatinné miesta
WindSense		
`wind_speed`	0,0 až 50,0 m/s	1 desatinné miesto
`wind_gust`	0,0 až 70,0 m/s	1 desatinné miesto
`wind_direction`	0 až 359 stupňov	celé číslo
`turbulence`	0,0 až 1,0	1 desatinné miesto
RainDetect		
`rainfall`	0,0 až 500,0 mm	1 desatinné miesto
`soil_moisture`	0,0 až 100,0 %	1 desatinné miesto
`flood_risk`	0 až 3	celé číslo
`rain_duration`	0 až 60 minút	celé číslo
AirQuality		
`co2`	300 až 5000 ppm	celé číslo
`ozone`	0,0 až 500,0 µg/m^3	1 desatinné miesto
`air_quality_index`	0 až 500	celé číslo

Pri `flood_risk` znamená 0 žiadne, 1 nízke, 2 stredné a 3 vysoké riziko. JSON používa desatinnú bodku; `NaN` a nekonečno sa odmietajú.

Integrita a časovanie

CRC32 sa počíta funkciou `zlib.crc32()` nad UTF-8 serializáciou celého objektu bez poľa `crc`. Kľúče sa zoradia abecedne aj vo vnorených objektoch a vynechajú sa medzery. Ne-ASCII znaky sa zapíšu únikovými sekvenciami; nečíselné hodnoty sa nepovolia. Poradie kľúčov a odsadenie prijatého JSON preto výsledok nemenia. Číselná reprezentácia sa zachováva: napríklad `60` a `60.0` sa líšia. Pri simulácii chyby sa mení iba odosielaná kópia až po výpočte CRC.

Prvá DATA sa odošle po registrácii, ďalšie každých 10 s. Ručné meranie nemení plán automatiky. Na ACK sa čaká 3 s; bez odpovede sa požiadavka zahodí. Automatické opakovanie pri strate ACK táto verzia neobsahuje.

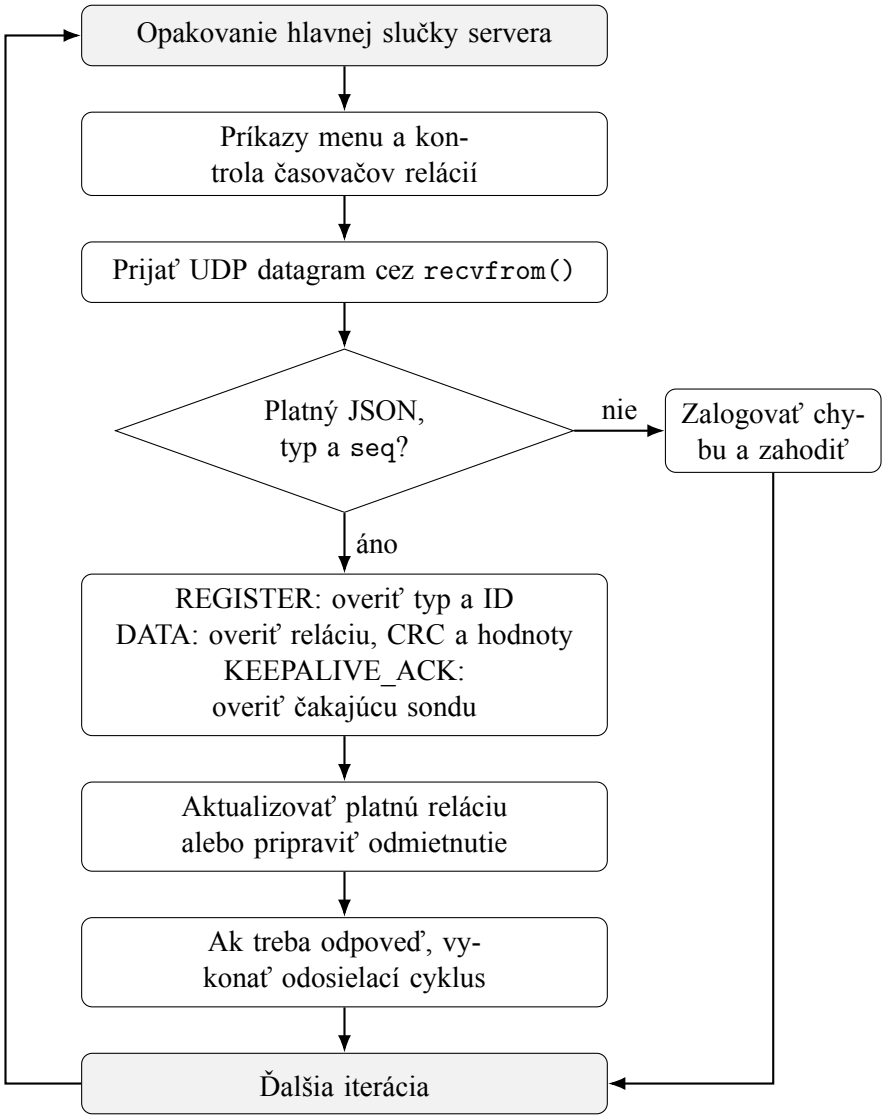
Po 15 s bez DATA server vypíše SENSOR IDLE a pošle KEEPALIVE. Bez odpovedí nasledujú sondy v 20. a 25. sekunde a zrušenie relácie v 30. sekunde s výpisom SENSOR DISCONNECTED. Ak DATA ešte neprišla, čas sa počíta od registrácie. KEEPALIVE_ACK vynuluje počet nezodpovedaných sond, nová DATA obnoví aktívny stav.

Lokálny časovač senzora je 30 s od platného kontaktu: KEEPALIVE, ACK merania alebo RETRANSMIT. Vypnutie automatiky nebráni odpovediam na sondy; úplné odpojenie áno. Po zániku relácie nová registrácia pridelí nový token. Časovanie používa `time.monotonic()`.

Komunikačné cykly

Prijímanie na serveri

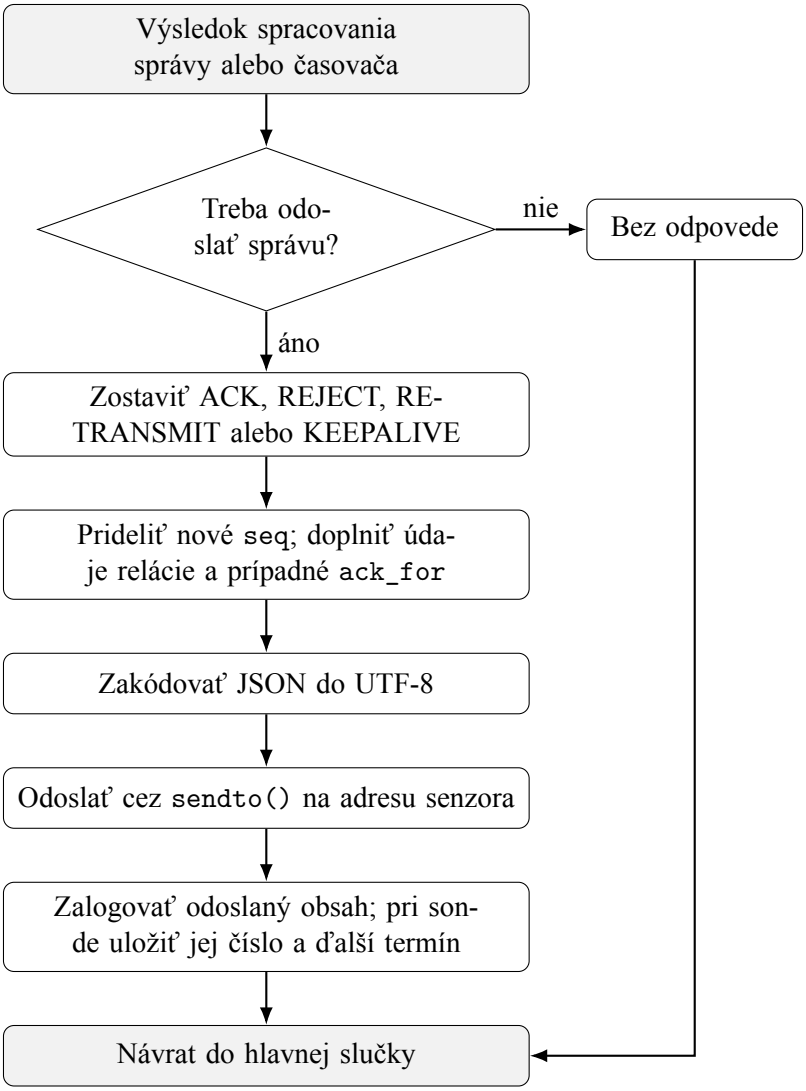
Server strieda obsluhu príkazov, časovačov a príjem UDP. Podľa typu správy overí registráciu, meranie alebo odpoveď na sondu a aktualizuje reláciu (obrázok 1). Neplatný JSON sa zaloguje a zahodí. Timeout socketu je najviac 0,2 s, aby kontroly fungovali aj bez prijatých správ.



Obrázok 1: Prijímací cyklus servera.

Odosielanie na serveri

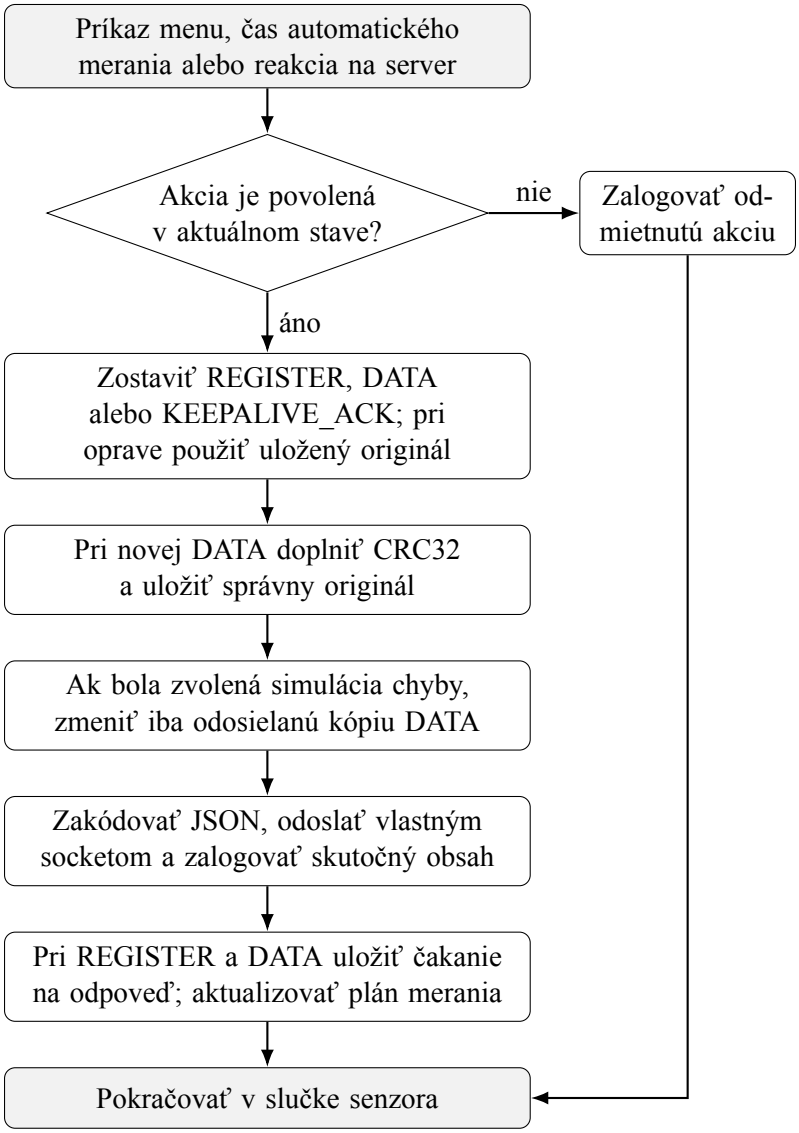
Odpoveď vzniká pri spracovaní správy alebo časovača. Server zostaví príslušný objekt, pridelí `seq`, zakóduje JSON a odošle ho na adresu senzora (obrázok 2). Pri KEEPALIVE uloží číslo sondy a termín ďalšej kontroly. Odosielanie je súčasťou hlavnej slučky.



Obrázok 2: Odosielací cyklus servera.

Odosielanie vo vlákne senzora

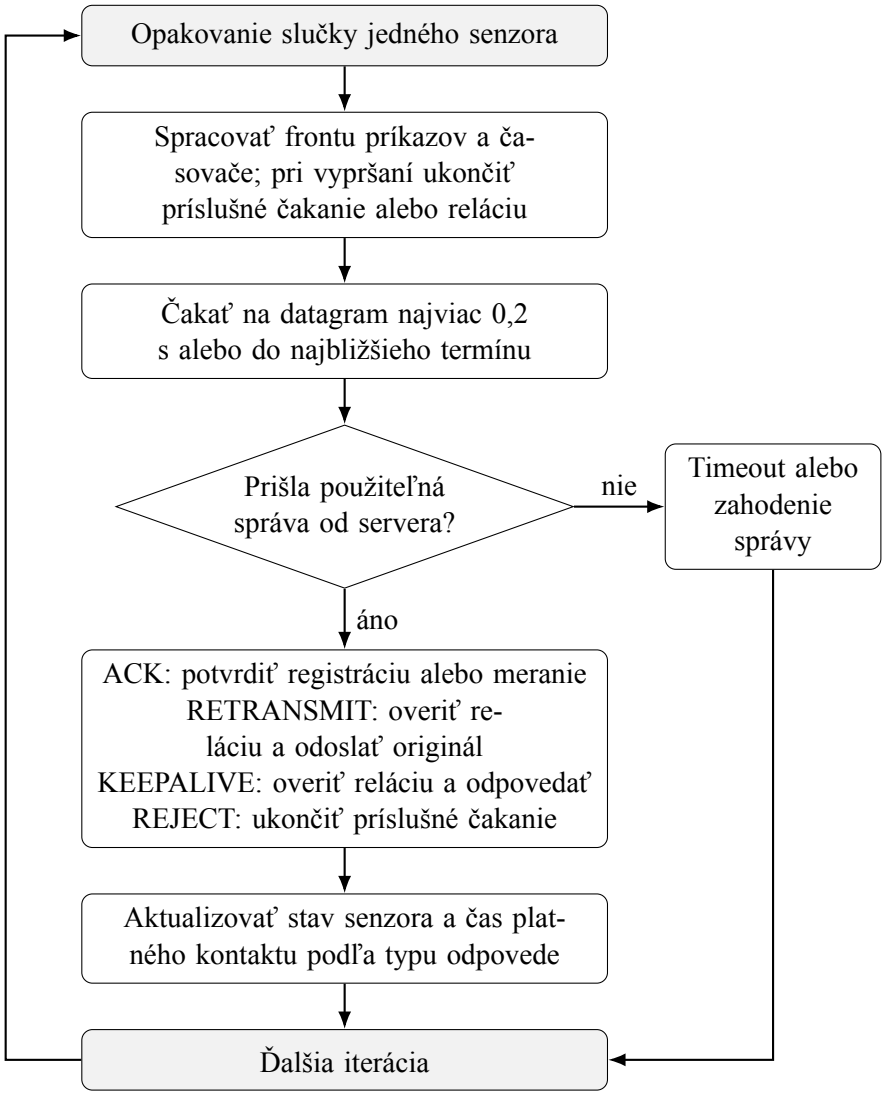
Vlákno spracuje príkazy svojej fronty a plán automatických meraní. Pri novej DATA doplní CRC a uchová správny originál. Pri retransmisii použije pôvodné číslo aj obsah (obrázok 3). Menu so socketom nepracuje.



Obrázok 3: Odosielací cyklus jedného senzora.

Prijímanie vo vlákne senzora

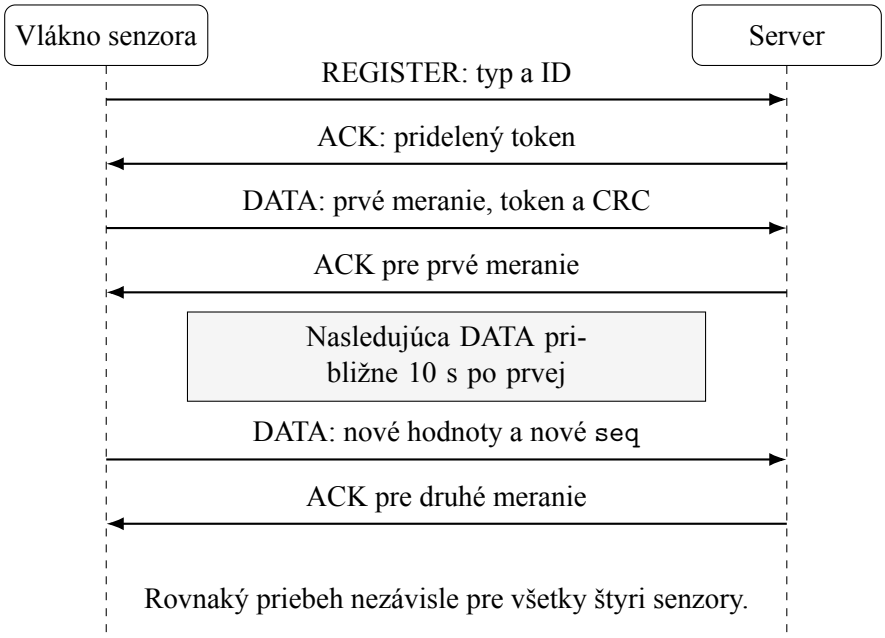
Senzor prijíma iba odpovede zo zadanej adresy servera. ACK ukončí čakanie, RETRANSMIT vyžiada odoslanie originálu a KEEPALIVE sa potvrdí aj pri vypnutých meraniach (obrázok 4). REJECT ukončí požiadavku; odmietnutie tokenu aj lokálnu reláciu. Pri ukončení testera sa zastavia všetky vlákna a zatvoria sockety.



Obrázok 4: Prijímací cyklus jedného senzora.

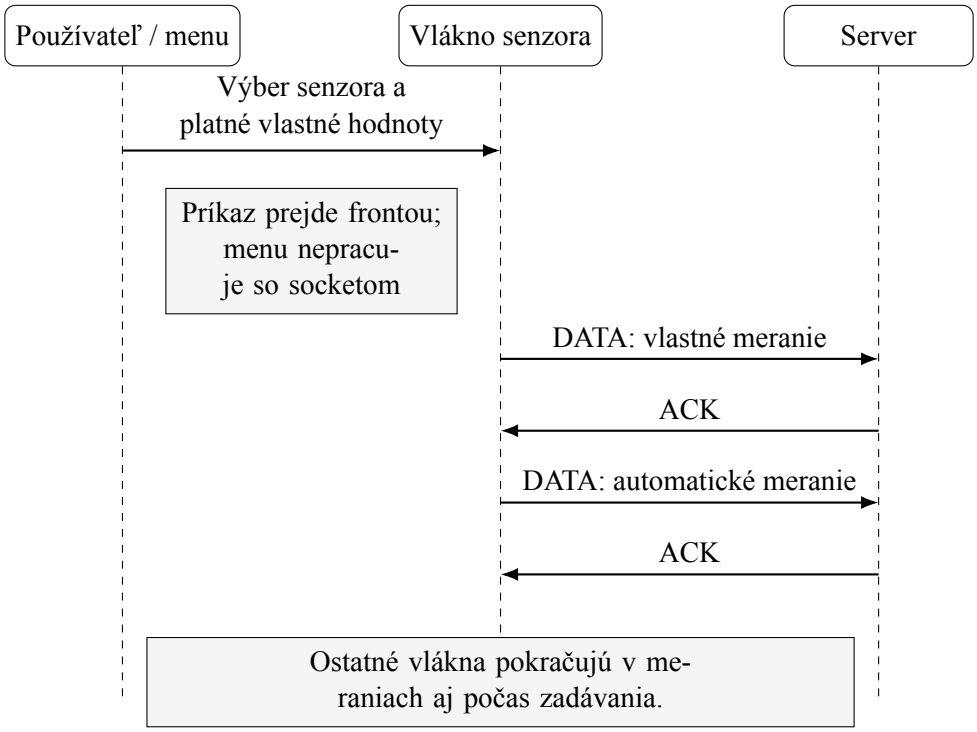
Sekvenčné diagramy a overenie UAT

UAT 1 – registrácia a automatické merania. Každý zo štyroch senzorov dostane token a odošle aspoň dve DATA s odstupom približne 10 s. Priebeh jedného senzora je na obrázku 5.



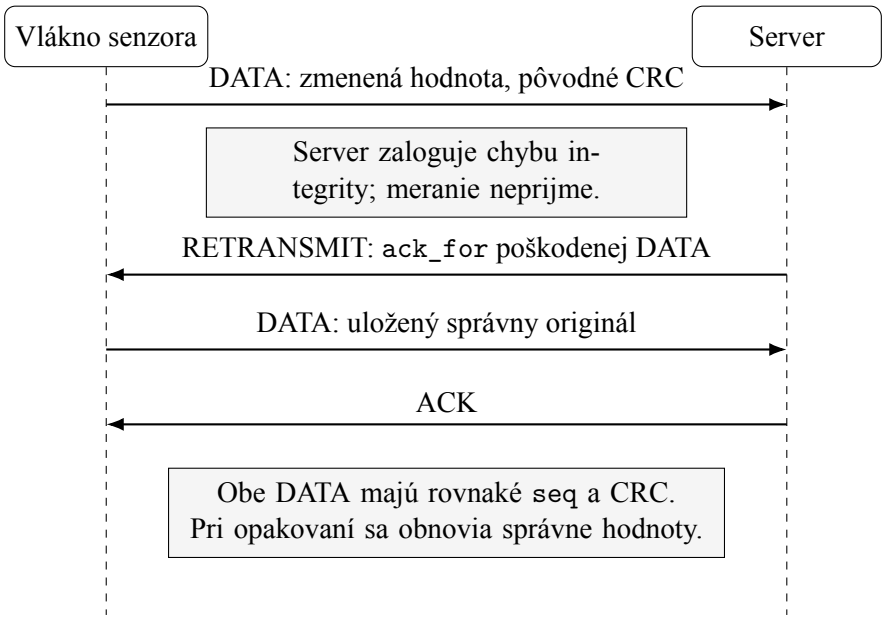
Obrázok 5: UAT 1: registrácia a dve automatické merania.

UAT 2 – vlastné meranie. Používateľ zadá hodnoty a server ich presne prijme a potvrdí. Automatické merania pokračujú aj počas zadávania (obrázok 6).



Obrázok 6: UAT 2: dodatočné meranie zadané používateľom.

UAT 3 – poškodenie správy. Tester poškodí nasledujúcu DATA. Server ju neprijme ako platné meranie a vyžiada originál. Opakovanie má rovnaké `seq` a CRC, ale obnovené správne hodnoty (obrázok 7).

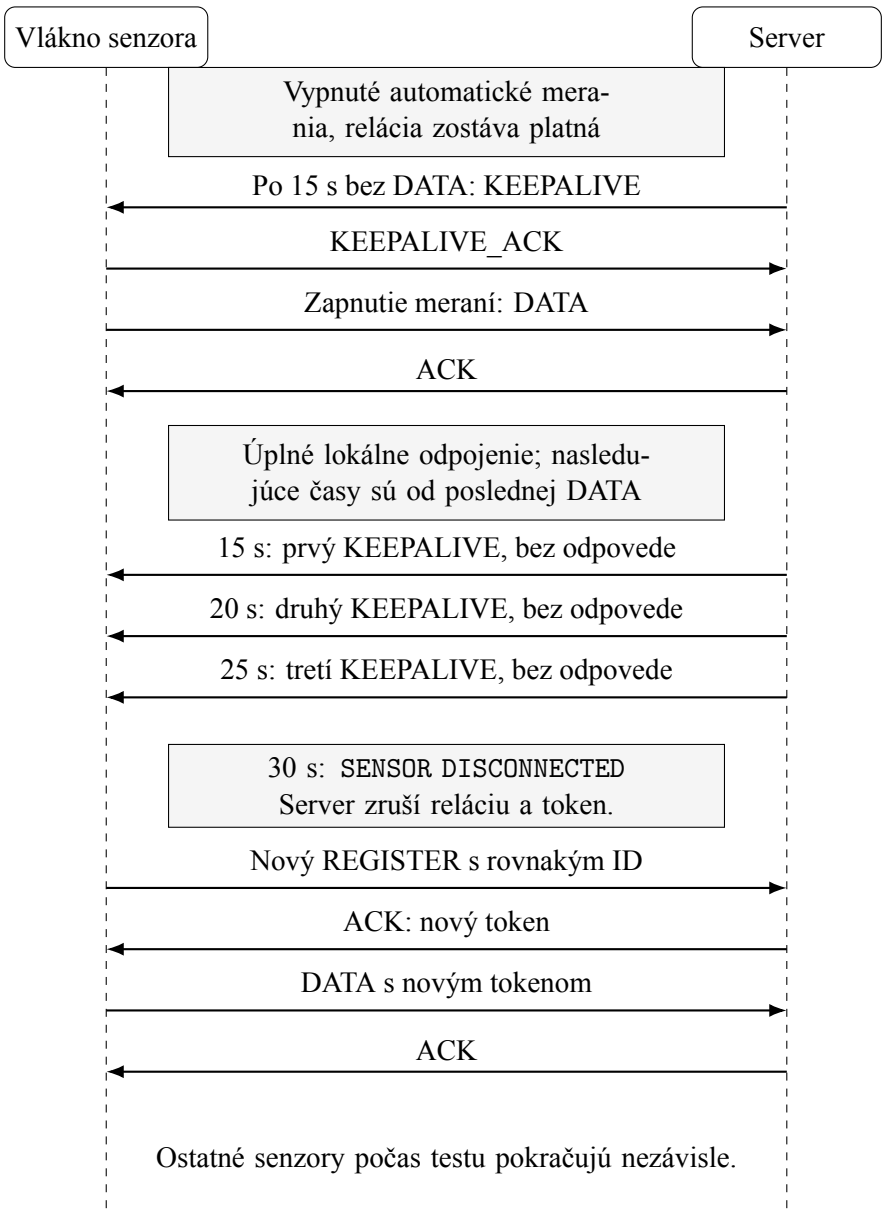


Obrázok 7: UAT 3: poškodenie a opakovanie správneho originálu.

Lokálne overenie. Prešlo všetkých 10 automatických testov vrátane UAT 1--4, chybnej IP a portu, hodnôt mimo rozsahu, presnosti vstupov a nezávislosti vlákien. Odpojenie bolo v testovanom behu zaznamenané po 30,001 s od posledného merania. Sieťové testy sa spúšťajú príkazom:

```
PKS_RUN_UDP_TESTS=1 python3 -m unittest discover -s tests -v
```

UAT 4 – dostupnosť a nové pripojenie. Najprv sa vypnú iba merania a overia odpovede na KEEPALIVE. Po obnovení jednej DATA sa senzor úplne odpojí. Server zruší reláciu po troch nezodpovedaných sondách a poslednej päťsekundovej lehote. Nová registrácia vytvorí nový token (obrázok 8).



Obrázok 8: UAT 4: nečinnosť, odpojenie a nová registrácia.

Návrh merania efektivity

JSON a budúci binárny formát sa porovnajú na rovnakej vopred uloženej sade hodnôt: štyri senzory, každý 60 meraní s intervalom 10 s. Započítajú sa registrácie aj potvrdenia, teda pri úspešnom základnom prenose spolu 488 datagramov. Oprava poškodenej správy a nečinnosť s KEEPALIVE sa vyhodnotia ako samostatné scenáre.

1. Zachytávanie vo Wiresharku sa spustí pred registráciou a filtruje sa podľa UDP portu servera, aby zahŕňalo všetky senzory.
2. Pre oba formáty sa použijú rovnaké merania, počty správ, sieťové podmienky a započítané vrstvy. Scenár sa vykoná aspoň trikrát.
3. Porovná sa počet datagramov a súčet bajtov na aplikačnej úrovni, L3 a L2. Výsledky sa zobrazia tabuľkou a grafom.

Pre IPv4 bez voliteľných polí a Ethernet II bez VLAN sa veľkosti vypočítajú ako:

$$L_{\text{L3}} = L_{\text{app}} + 20 + 8, \qquad L_{\text{L2}} = \max(64, L_{\text{app}} + 14 + 20 + 8 + 4).$$

Započítavajú sa hlavičky IPv4 a UDP, ethernetová hlavička, FCS a prípadná výplň. Ak FCS v zázname chýba, doplní sa analyticky a uvedie sa to pri výsledkoch. Pre Wi-Fi alebo loopback sa použije ich skutočná réžia. ACK a ostatné riadiace správy sa nesmú vynechať.

Úspora sa vyjadrí pre rovnakú vrstvu:

$$U = 100 \left(1 - \frac{B_{\text{binary}}}{B_{\text{JSON}}}\right) \%.$$

Pre ilustráciu má uvedená DATA ThermoNode po serializácii 168 B; s IPv4 a UDP 196 B a v tomto ethernetovom modeli 214 B. Dátové ACK s `seq=2` a `ack_for=2` má na L2 80 B, spolu teda 294 B na jedno potvrdené meranie. Ide o analytický výpočet konkrétneho príkladu, nie o výsledok merania binárnej verzie.

Zhodnotenie

Implementácia pokrýva UAT 1--4 a lokálne overuje registráciu, prenos vlastných hodnôt, opravu poškodenia a správu relácií. Oddelené vlákna umožňujú nezávislé správanie senzorov. JSON je čitateľný, ale jeho objem je vyšší než pri cielenej binárnej reprezentácii; rozdiel sa vyhodnotí uvedeným postupom. Aktuálny tester používa štyri pevne nastavené senzory a pri strate ACK správu automaticky neopakuje.